Implementing GKR from first Principle

Anjul kc

May 2026

What is this about?

Before diving into the implementation details, it is important to understand what a proof is and the fundamental design principles behind zero-knowledge (ZK) systems. I strongly recommend that the reader first develops a clear understanding of the **Sum-Check** and **GKR** protocols. **Thaler's** manuscript is an excellent resource for building intuition about proofs, arguments, and zero-knowledge constructions.

The primary objective of this documentation is to bridge the gap between the highly abstract mathematics underlying the GKR protocol and its practical implementation. In particular, it focuses on how to design appropriate data structures and highlights important considerations during the construction process. The material will be significantly easier to follow if the reader has worked through at least one simple example. If not, I recommend starting with a step-by-step example available **here**.

Throughout this document, I will connect the mathematical formulations with their corresponding implementation steps, showing how each concept translates into code. The implementation is written in C++, as I prefer building systems from first principles.

While working through the GKR protocol on paper is relatively straightforward, implementing it in code introduces a level of complexity that is often underestimated. This documentation primarily focuses on the prover's task of convincing a verifier that it has correctly evaluated an n-degree univariate polynomial of the form

$$a_0 + a_1 x + a_2 x^2 + \cdots + a_n x^n.$$

The implementation approach presented here is flexible and may vary depending on the reader's understanding. Writing imperfect or messy code is completely acceptable at this stage—the key objective is to connect the underlying concepts.

The implementation is divided into several key steps. First, we construct an arithmetic circuit, modeled as a binary tree in which each internal node has two incoming edges. Next, we apply Breadth-First Search (BFS) to assign a binary label to each gate (node). A second BFS traversal is then used to initiate the GKR protocol, during which we collect essential information such as gate values at each layer, node labels, child relationships, and the corresponding gate operations (addition or multiplication).

At first glance, it may seem unnecessary to use the GKR protocol when a verifier could simply request the value of x from the prover and verify the result directly, which would be faster. However, the purpose of GKR is not just efficiency in this trivial setting—it serves as a powerful framework for understanding and implementing the Sum-Check protocol. In fact, this example demonstrates a simplified setting that captures the essential ideas enabling GKR to function.

Since GKR is a public-coin interactive proof (IP), this documentation will also explain how to transform the interaction into a non-interactive protocol using the Fiat–Shamir heuristic.

Overall, this document reflects my personal approach to understanding and implementing the GKR protocol. While the code may not be perfect, it captures the key components required to build a working implementation.

A proof is anything that convinces someone that a statement is true and a "proof

system" is any procedure that decide what is and is not a convincing proof. Any proof system must follow this two property:

- Any true statement should have a convincing proof of its validity. This property is typically referred to completeness.
- No false statement should have a convincing proof. This property is typically referred to soundness.

When it comes to ZK(Zero knowledge) design the system is typically developed into two step process. First an Information-theoretically secure protocol, such as an IP, multi-prover interactive proof(MIP) or Probabilistically checkable proof(PCP) is developed for a model involving one or more provers that are assumed to behave in some restricted manner. Second the information-theoretically secure protocol is combined with cryptography to "force" a single prover to behave in the restricted manner, thereby yielding an argument system. This second step also often endows the resulting argument system with important properties such as ZK, succinctness and non-interactivity. If the resulting argument satisfies all of these properties, then it is infact a ZK-SNARK.

Building our Arithmetic Circuit

Throughout this documentation, we use the public input set $\{3, 10, 16, 15, 1, 9\}$, which represents the coefficients of a degree-5 polynomial $3 + 10x + 16x^2 + 15x^3 + x^4 + 9x^5$, evaluated at $x = 17$. All computations are performed over a finite field with prime modulus $P = 67$. While working over a finite field is not strictly required, avoiding it quickly leads to large intermediate values, making implementation cumbersome. In practice, it is sufficient to understand modular arithmetic and its behavior.

One important detail to note is the witness value ($x = 17$), which is known to both the prover and the verifier. In the GKR protocol, both parties must have a shared understanding of the circuit structure. Therefore, this setting is not zero-knowledge; rather, it is a standard interactive proof. There exist more structured and efficient ways to model such computations, most notably through **Rank-1 Constraint Systems (R1CS)**. Although R1CS is beyond the scope of this document, an interested reader can explore how the interaction can be expressed using the sum-check protocol applied to

$$A \cdot w \odot B \cdot w - C \cdot w = 0,$$

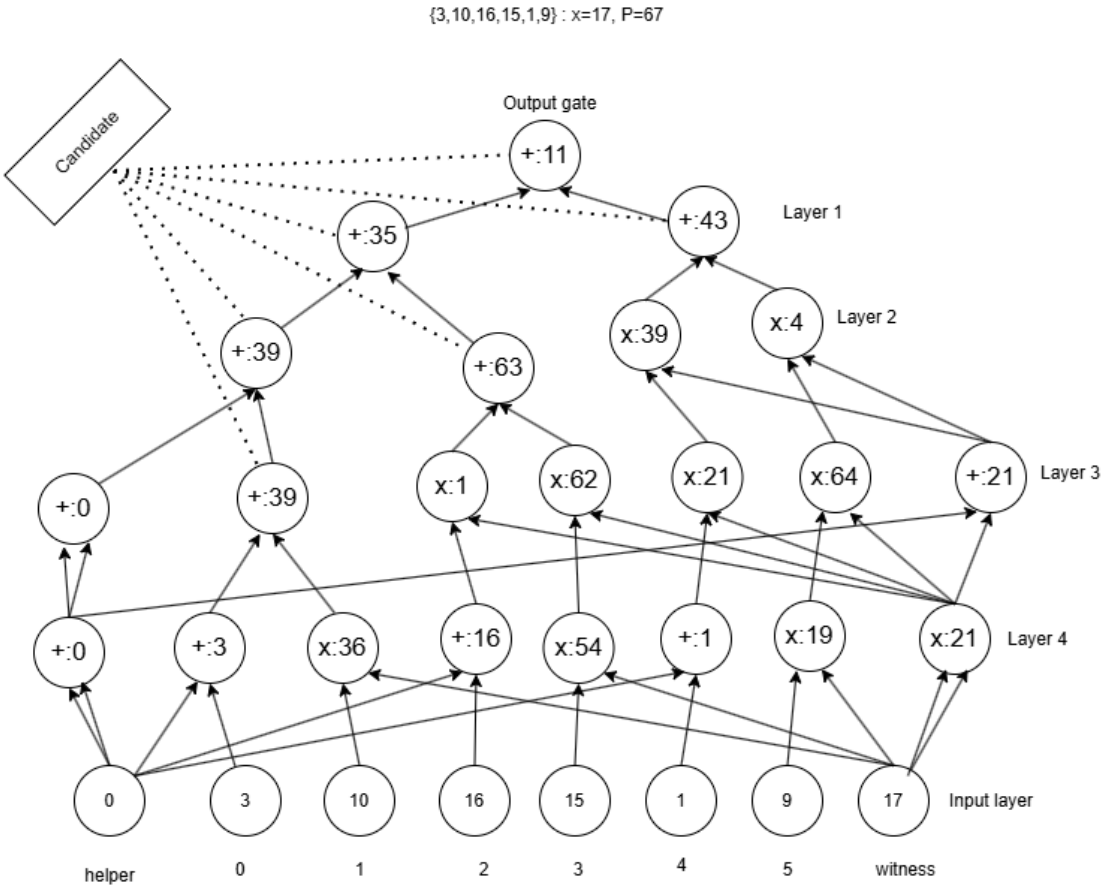
where the initial claim ie. sum of evaluations of a polynomial over all Boolean input is zero. This formulation also serves as a strong foundation for integrating polynomial commitment schemes.

For readers with a solid background in group theory, finite fields, and algorithms such as the Number Theoretic Transform (NTT), there is a rich landscape to explore in zero-knowledge systems. In particular, polynomial IOPs offer a powerful generalization—replacing binary labels with subgroup elements and applying univariate sum-check leads to remarkably elegant constructions. There is much more depth to these topics, but a detailed exploration would take us far beyond the scope of this

document.

The Construction:

There are two primary approaches for constructing the circuit (which we model as a tree): a top-down approach and a bottom-up approach. The top-down approach begins at the root, representing the output gate, and proceeds toward the input layer (the leaves). However, this approach is less practical in our setting, as it requires managing intermediate values without having them readily computed, especially for individual addition and multiplication operations. In contrast, the bottom-up approach offers better control over the computation. It starts from the leaves and progresses toward the root by explicitly computing intermediate results at each step. This makes the process more structured and easier to manage. A consistent workflow can be achieved by first computing the parent nodes of the leaves and then iteratively proceeding upward until reaching the root. The figure below illustrates the corresponding tree structure used in our implementation.



Assume the input $\{3, 10, 16, 15, 1, 9\}$ are stored in an array, where each index corresponds to the degree of the polynomial. We construct a node for each element in this array. In addition, we introduce one node for the witness ($x = 17$) and a helper node (with value 0 or 1). If the helper node is assigned the value 1, then instead of addition we need to use multiplication. The primary purpose of the helper node is to enforce the structural constraint that the circuit must have fan-in two. The GKR

protocol operates on arithmetic circuits where each gate has exactly two inputs, so we cannot arbitrarily connect nodes across layers (e.g., directly from layer 4 to layer 2). This restriction will become clearer later. If we examine the construction carefully, we observe a consistent pattern above layer 4, with Layer 4 being slightly different.

Layer 4 is constructed by multiplying values at odd indices with the witness, while values at even indices are added with the helper node. As we proceed upward, the helper node is propagated, and the witness is squared once . The reasoning behind this is straightforward: any odd degree can be expressed as $2n + 1$. By multiplying odd terms with the witness, we enable their parent nodes to be consistent with square value of witness.The helper is propagated at least once. The pattern works for arbitrary odd degree polynomial. It is totally upto the reader to determine the pattern for even degree polynomial, and make either choice. We can actually model the above circuit for even degree simply by appending a 0 in array. At this point the work for layer 4 is completed and we can make use of the recursion to build the tree.

Pattern: To initiate the recursive construction, we store the nodes of Layer 4 in a data structure that supports constant-time access to the front element. For this purpose, we use a queue. At each step, the first two elements are dequeued and combined using an addition operation, producing a new node. This node is then added to a candidate container. The remaining elements in the queue are each multiplied by the square of the witness, producing new nodes via multiplication. These nodes are inserted into a new queue for the next recursion. At this stage, we must decide whether the witness and helper nodes need to be extended.

The rule is straightforward: if the size of the new queue is greater than two, the witness must be extended. This extension is performed by combining the squared witness with the helper node using an addition operation, resulting in a new node. The intuition behind this process is as follows. In each recursive step, two elements are removed from the queue and combined, while the remaining elements must be adjusted (via multiplication) to maintain consistency with the circuit structure. However, if only two elements remain, no further multiplication is necessary, and thus no extension of the witness is required.

However, extending the helper node requires more careful handling. To understand this, we first clarify the role of the *candidate container*. Recall that at each step, we dequeue two elements from the queue and combine them using an addition operation to form a new node. This newly created node is then inserted into the candidate container. The candidate container can hold at most two elements. If the candidate contains two elements, they are immediately combined via addition to produce a single node, which is then stored back in the candidate container. This ensures that the container maintains a running aggregation. If the candidate contains only one element, we encounter a structural inconsistency. In this case, we combine the existing element with the helper node, producing a new node via addition, and insert the result back into the candidate container. The elements within the candidate container are always combined using addition, as the container represents the running sum of terms of the form $a_i x^i$. The case where the container size becomes 1 corresponds to an intermediate imbalance—for example, at the initial stage where the polynomial

begins as $a_0 + a_1 x$, which does not yet fully align with the general structure $a_i x^i$. The helper node resolves this imbalance and preserves the uniformity of the construction.

After understanding the notion of *candidate container*. The helper is extended under two conditions:

- when the candidate container has size one, or
- when the size of the new queue (formed after multiplication with the squared witness) exceeds four.

To understand this, consider the process at Layer 3. Suppose the new queue contains four elements (which will form Layer 2). In this case, we extend the witness (since the queue size is greater than two), but we do not extend the helper. The existing helper from Layer 3 is sufficient. However, if the new queue contains six elements, the situation changes. In this case, we must extend both the witness and the helper. The reason is that, in the next step (Layer 2), the queue will reduce to four elements, and extending the witness at that stage will require a corresponding helper node. Importantly, we cannot reuse the helper from Layer 3, as each layer requires its own consistent extension to maintain the circuit structure.

This distinction ensures that the recursive construction remains well-formed and satisfies the fan-in two constraint at every layer. Observe that the depth of the circuit is directly proportional to the highest degree of the polynomial. For simplicity, one could adopt an alternative approach by precomputing all powers of the witness and directly combining each coefficient with its corresponding power. This results in a circuit with a single multiplication layer followed by layers consisting only of addition operations. In fact, I initially used this approach to construct a simpler circuit and test my GKR implementation. However, at the end of this document, I will explain what breaks. But our primary concern is GKR so reader can happily use this second approach. The more complex circuit requires careful handling during BFS traversal, primarily because the witness and helper nodes act as in-neighbors for many other nodes. Conceptually, this is similar to having multiple owners of the same memory resource. To manage this safely in code, I use **shared_ptr**. Since multiple nodes reference the same underlying object (e.g., the witness or helper), shared ownership is necessary. A `shared_ptr` ensures that the object remains alive as long as at least one reference exists, and automatically deallocates the memory once all owners go out of scope. Now lets see the code

Data Structure

```
enum OPERATOR{ADD,MUL,NONE};
class Node{
public:
    int value;
    string label;
    shared_ptr<Node>left;
    shared_ptr<Node>right;
    OPERATOR op;
    bool helper,witness;
    Node(){}
    Node(int value):op(NONE),helper(false),witness(false){
```

```
        this->value=value;
    }
    Node(OPERATOR op,shared_ptr<Node>left,shared_ptr<Node>right
        )
    :helper(false),witness(false){
        this->op=op;
        switch (op)
        {
            case MUL:
                value=(left->value* right->value)%Prime;
                break;
            case ADD:
                value=(left->value + right->value) % Prime;
                break;
            case NONE: break;
        }
        this->left=left;
        this->right=right;
    }
};
```

bool helper, witness is simply indicator whether a node is helper or witness and it will be handy later on when using BFS, which ensure that we visited once these node. Label store the binary label that the corresponding gate gets.

Circuit Construction

```
shared_ptr<Node> makeCircuit(const vector<int>&x,const int& w)
{
    std::shared_ptr<Node>helper=std::make_shared<Node>(0);
    std::shared_ptr<Node>witness=std::make_shared<Node>(w);
    helper.get()->helper=true, witness.get()->witness=true;
    vector<shared_ptr<Node>>input(x.size());
    for(size_t i=0;i<x.size();i++) input[i]=std::make_shared<
        Node>(x[i]);
    std::queue<shared_ptr<Node>>parent;
    for(size_t i=0;i<input.size();i++) {
        if( i & 1) parent.push(std::make_shared<Node>(MUL,input
            [i],witness)); // Odd
        else parent.push(std::make_shared<Node>(ADD,helper,
            input[i])); // Even
        input[i].reset();// release the owned object
    }
    std::shared_ptr<Node>helper_chain=std::make_shared<Node>(
        ADD,helper,helper);
    shared_ptr<Node>witness_chain=std::make_shared<Node>(MUL,
        witness,witness);
    helper.reset(),witness.reset();// release
    helper_chain.get()->helper=true,witness_chain.get()->
        witness=true;
    return Circuit::RecursiveBuild(parent,witness_chain,
        helper_chain,{});
```

```
}
```

Recursive Build

```
   //build the tree from bottom up to  make circuit
static shared_ptr<Node>RecursiveBuild(std::queue<shared_ptr<
   Node>>parent,shared_ptr<Node>witness,
shared_ptr<Node>helper,vector<shared_ptr<Node>>candidate)
{
    //ADD candidate
    if(candidate.size() == 1){
        shared_ptr<Node>node=std::make_shared<Node>(ADD,helper,
           candidate[0]);
        candidate.pop_back();
        candidate.push_back(node);
    } else if(candidate.size()==2) {
        shared_ptr<Node>node=std::make_shared<Node>(ADD,
           candidate[0],candidate[1]);
        candidate.pop_back(),candidate.pop_back();
        if(parent.empty()) return node;
        candidate.push_back(node);
    }
    auto left=parent.front(); parent.pop();
    auto right=parent.front(); parent.pop();
    shared_ptr<Node>root=std::make_shared<Node>(ADD,left,right)
       ;
    candidate.push_back(root); //push to the add candidate
    std::queue<shared_ptr<Node>>ancestor;
    while(!parent.empty()){
        auto top=parent.front();
        parent.pop();
        ancestor.push(std::make_shared<Node>(MUL,top,witness));
    }
    if(ancestor.size() > 2) { //witness chain
        shared_ptr<Node>node=std::make_shared<Node>(ADD,helper,
           witness);
        node.get()->witness=true;
        witness=node;
    }
    if(candidate.size()==1 || ancestor.size() - 2 > 2) { //
       helper chain
        shared_ptr<Node>helper_chain=std::make_shared<Node>(ADD
           ,helper,helper);
        helper_chain.get()->helper=true;
        helper=helper_chain;
    }
    return RecursiveBuild(ancestor,witness,helper,candidate);
}
```

The base case for the recursion occurs when the queue is empty. As described earlier, the candidate container is maintained such that it ultimately holds two elements, which are combined during each recursive step.When no elements remain in the queue,

the final value stored in the candidate container represents the root of the circuit.

Main function

```cpp
int main()
{
    int witness=17;
    vector<int>input={3,10,16,15,1,9};
    shared_ptr<Node>root=makeCircuit(input,witness);
    Circuit::AssignLabel(root.get());
    auto r=BFS(root.get());
   //verifier can check in her own
   //let's assume prover sends the witness along side.
   vector<int>temp;
   temp.reserve(2+input.size());
   temp.push_back(0);
   for(auto &x:input) temp.push_back(x);
   temp.push_back(witness);
   std::cout<<"\n"<<LabelInformation::evaluateW(temp,r);
    return 0;
}
```

The entry point of the program. For now, functions such as `AssignLabel`, `BFS`, and `evaluateW` can be ignored—they will be explained in detail in later sections as we develop the components required for the GKR protocol. One important notation detail is the use of `ClassName::FunctionName`. For example, `Circuit::` denotes a static member function of the `Circuit` class, and `LabelInformation::` denotes a static member function of the `LabelInformation` class. I prefer this style because it helps organize the code by clearly separating concerns while still keeping related functionality grouped logically. Additionally, since these functions do not rely on member variables, using static functions avoids unnecessary object instantiation and keeps the design simple.

OUTPUT

```
11,
35, 43,
39, 63, 39, 4,
0, 39, 1, 62, 21, 64, 21,
0, 3, 36, 16, 54, 1, 19, 21,
0, 3, 10, 16, 15, 1, 9, 17,
Claimed value of Output gate: 11
```

This is the output circuit, if used BFS traversal. It matches the above figure. Reader of this documentation are suggested to find an edge case(degree), where the circuit construction breaks, with this Our Arithmetic Circuit construction is finished.

The GKR protocol verifies the claim $C(x, w) = y$ layer by layer with a different instance of Sum-check protocol required for each layer of C.

The GKR protocol

This section is going to be long, where i will try to cover what GKR protocol, how it works and some mathematical construction along with the coding part.

Goldwasser, Kalai and Rothblum (GKR) descirbed a remarkable interactive protocol that is best presented in terms of arithmetic circuit evaluation problem. In this problem verifier(V) and prover(P) first agrees on a log-space uniform arithmetic circuit C of fan-in 2 over a finite field F, and the goal is to compute the value of the output gates of C. A log space uniform circuit C is one that possesses a succinct implicit description, in a sense that there is a logarithmic-space algorithm that takes as input the label of gate a of C and is capable of determining all relevant information about the gate. That is simply outputting the labels of all of a's neighbor and determining if a is an addition or multiplication gate.

C is assumed to be in layered form, meaning that the circuit can be decomposed in layers and wires only connected gates in adjacent layers(ie. layer 2 cannot have its neighbor from layer 4). Suppose circuit has depth d and number the layer from 0 to d with layer d referring the input layer and layer 0 referring the output layer. In the first message P tells verifier the claimed output of the circuit. The protocol then works its way in iterations toward the input layer, with one iteration devoted to each layer. The purpose of iteration i is to reduce a claim about the values of the gate at layer i to a claim about the values of the gates at layer i+1, in a sense that it is safe for V to assume that the first claim is true as long as second claim is true. This reduction is accomplished by applying sum-check protocol.

What that reduction means
The GKR protocol starts with a claim about the values of the output gates of the circuit, but verifier cannot checks this claim without evaluating the circuit itself. So the first iteration uses a sum-check protocol to reduce this claim about the output of the circuit to a claim about the gate value at layer 1. Once again V cannot check this claim itself. So the second iteration uses another sum-check protocol to reduce the latter claim about the gate value at layer 2 and soon. Eventually V is left with a claim about the inputs to the circuit and V can check this claim without any help.

Let S_i denotes the number of gates at layer i of the circuit. The GKR protocol make use of several functions, each of which encodes certain information about circuit. Note functions are not necessarily computable. We define our first function $W_i : \{0,1\}^{k_i} \rightarrow F_{67}$. W_i is a mapping function that takes a binary gate label and maps to the gate value for layer i. k_i simply is the number of bits required to uniquely label the each gates in layer i. Now it is a right moment to introduce about Multilinear extension polynomial (MLE).

A multivariate polynomial is said to be multilinear if the degree of the polynomial

in each variable is at most 1. A v-variate polynomial g over Field is said to be an extension of W if g agrees with W at all Boolean-valued inputs ie. g(x)=W(x) for all $x \in \{0,1\}^v$. Let's use value from layer 3. The number of gates at layer 3 is 7 and 3 bits is enough to represent number from 0-6 uniquely given as ceil($log_2(S_i)$). We can define a mapping as

$$W(0,0,0) \rightarrow 0$$
$$W(0,0,1) \rightarrow 39$$
$$W(0,1,0) \rightarrow 1$$
$$W(0,1,1) \rightarrow 62$$
$$W(1,0,0) \rightarrow 21$$
$$W(1,0,1) \rightarrow 64$$
$$W(1,1,0) \rightarrow 21$$

This suggests that each gate value must be assigned a unique binary label. We begin by constructing these labels. If you are familiar with binary pattern generation, the same idea can be applied here. Alternatively, the labeling can be integrated directly into the circuit construction process. In my implementation, I use a Breadth-First Search (BFS) traversal to collect all gates at layer i, and then assign binary labels to those gates.

BFS used for assigning label

```
static void AssignLabel(Node* root){
//inside Circuit class
    std::deque<Node*>queue;
    queue.push_back(root);
    size_t size=1;
    while(size > 0){
        shared_ptr<Node>witness=nullptr;
        shared_ptr<Node>helper=nullptr;
        while(size--){
            auto top=queue.front();
            queue.pop_front();
            if(top->left) {
                if(top->left.get()->witness) continue;
                if(top->left.get()->helper) helper=top->left;
                else queue.push_back(top->left.get());
            }
            if(top->right){
                if(top->right.get()->helper) continue;
                if(top->right.get()->witness) witness=top->
                   right;
                else queue.push_back(top->right.get());
            }
        }
        if(helper) queue.push_front(helper.get());
        if(witness) queue.push_back(witness.get());
        vector<Node*>values(queue.begin(),queue.end());
        iterator=0;
        assignLabel(values,"",ceil(log2(values.size())));
        size=queue.size();
```

```
    }
}
```

This is a standard BFS traversal. However, instead of using a queue, we use a `deque`. The reason is that the helper node may need to be inserted at the front when processing a layer i. A `deque` naturally supports efficient insertion at both the front and back, making it a suitable choice. The `iterator` is implemented as a static member variable, whose sole purpose is to reset the indexing (values) to 0. Additionally, flags for the witness and helper nodes are used to ensure that each of these nodes is inserted into the deque at most once per layer. This prevents duplication and maintains consistency in the traversal.

assignLabel function definition

```
static void assignLabel(vector<Node*>&values,string label,int
   depth){
    if(iterator >= values.size()) return;
    if(depth<=0){
        values[iterator]->label=label;
        iterator++;
        return;
    }
    label.push_back('0');
    assignLabel(values,label,depth-1);
    label.pop_back();
    label.push_back('1');
    assignLabel(values,label,depth-1);
    label.pop_back();
}
```

I don't think this part needs an explanation, depth is the number of bit required and we are stopping earlier if iterator equals the size of values. vector<Node*>values store all of the nodes(gates) at layer i.

Updated circuit information

```
11: ,
35: 0, 43: 1,
39: 00, 63: 01, 39: 10, 4: 11,
0: 000, 39: 001, 1: 010, 62: 011, 21: 100, 64: 101, 21: 110,
0: 000, 3: 001, 36: 010, 16: 011, 54: 100, 1: 101, 19: 110, 21: 111,
0: 000, 3: 001, 10: 010, 16: 011, 15: 100, 1: 101, 9: 110, 17: 111,
```

If we have multiple output gate then each will get their labels and the process begins by prover sending MLE of output gate instead of claim value. The verifier evaluate MLE at random and set the claim and rest of the process is devoted to prove this claim.

MLE

Returning to our mapping, we now define the first 3-variate multilinear extension (MLE) for Layer 3. Determining the number of variables is straightforward: it is equal to the number of bits required to uniquely label the gates within that layer.

The multilinear extension corresponding to the gate values at Layer 3 is given by $\widetilde{W}(x_0, x_1, x_2) = 39(1-x_0)(1-x_1)x_2 + (1-x_0)x_1(1-x_2) + 62(1-x_0)x_1x_2$

$$+21x_0(1-x_1)(1-x_2) + 64x_0(1-x_1)x_2 + 21x_0x_1(1-x_2) \tag{1}$$

We have extended W. If you are unsure you can take input from $\{0,1\}^3$ and evaluate $\widetilde{W}$ and check that it is consistent with the mapping for example, if i pick $\{1,0,1\}$ then $\widetilde{W}(1,0,1) = 64 * 1 * 1 * 1 = 64$.

Now the question is how to define Multilinear extension ?
The expression for Multilinear Extension(MLE) is given by Lagrange Interpolation. It is defined as: let $f : \{0,1\}^v \to F$ be a function then the following multilinear polynomial $\widetilde{f}$ extends f:
$\widetilde{f}(x_1, \cdots, x_v) = \sum_{w \in \{0,1\}} f(w) \cdot \lambda_w(x_1, \cdots, x_v)$
where for any $w = (w_1, \cdots, w_v)$,
$\lambda_w(x_1, \cdots, x_v) = \prod_{i=1}^{v}(x_i.w_i + (1-x_i)(1-w_i))$
which is refereed to as the set of multilinear Lagrange basis polynomials.
It is clear that the x_i's represent the variables of the multilinear extension, but what does w represent? The value w corresponds to a binary label assigned to a gate. These binary labels ensure that the multilinear extension satisfies the property

$$\widetilde{f}(x) = f(x) \quad \text{for all } x \in \{0,1\}^v.$$

The underlying Lagrange basis polynomials make this behavior natural. When $x = w$, the basis polynomial associated with the label w evaluates to 1, while all other basis polynomials evaluate to 0. As a result, the multilinear extension evaluates precisely to $f(w)$. This is directly analogous to the behavior of the univariate Lagrange basis, except that the construction is extended to the multilinear setting over Boolean hypercube points.

This property is extremely powerful. Once we construct the multilinear extension (MLE), the verifier gains the flexibility to evaluate the function at any random point $x \in \mathbb{F}^v$, rather than being restricted only to points on the Boolean hypercube. This ability to move from Boolean inputs to arbitrary field elements is one of the key ideas underlying the sum-check protocol and interactive proofs in general.
The MLE alone is not that much meaningful and in real programming evaluating MLE is analogous to generating binary pattern. Actually in programming we will not spend our time building MLE, We will be directly going to evaluate MLE at given points. The underlying protocol used is Sum-check to prove the claim layer-by-layer. So It makes quite easier to work with MLE.

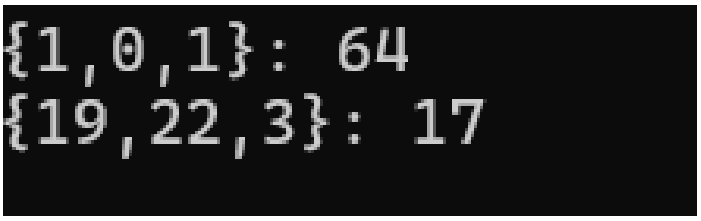
Program to evaluate MLE

```
1 static int evaluateW(const vector<int>& W, const vector<int>&
    points) {
2     //W is array of gate value at layer i.
3     vector<int>intermedies; // stores basis values
4     intermedies.reserve(W.size()); //points size is equal to
        the label which is no. of variables
5     evaluateW(points,0,points.size(),1,intermedies);
```

```
    int result=0;
    for(size_t i=0;i<W.size();i++){
        result=(result + W[i]*intermedies[i])% Prime;
    }
    return result;
}
```

```
static void evaluateW(const vector<int>&points,int i, int depth
    ,int val,vector<int>&result){
    if (depth <= 0){
        result.push_back(val);
        return;
    }
    int left= 1- points[i];
    if(left < 0) left+=Prime;
    left=(left * val) % Prime;
    evaluateW(points,i+1,depth-1,left,result);
    int right=(points[i]* val) % Prime;
    evaluateW(points,i+1,depth-1,right,result);
}
```

Evaluation of MLE of eqn(1)



The program computing the Lagrange basis polynomials is analogous to binary pattern generation. The term $(1 - \texttt{points}[i])$ corresponds to generating a binary 0, while $\texttt{points}[i]$ corresponds to generating a binary 1. However, instead of using strings to represent binary patterns, we use an integer variable `val` to accumulate the product terms. The parameter `depth` represents the number of variables, which is equivalent to the number of bits required for the binary labels at a particular layer. The array `W` stores the values of $f(w)$. Its ordering is extremely important because the intermedies store the Lagrange basis polynomials sequentially. For example, if $f(0,0,0) \to 0$, then the value 0 must appear in `W` at index 0, corresponding to the binary label $(0,0,0)$. All computations are performed modulo 67. When we are working in layer i the `W` have the gate values of layer i+1.

The GKR protocol also makes use of the notion of a "wiring predicate" that encodes which pairs of wires from layer i+1 are connected to a given gate at layer i in C. Let $in_{1,i}, in_{2,i} : \{0,1\}^{k_i} \to \{0,1\}^{k_{i+1}}$ denote the function that takes as input the label a of a gate at layer i of C and respectively output the layer of the first and second in-neighbor of gate a. It is not important in our case. Our interest is actually two function add_i and $mult_i$ mapping $\{0,1\}^{k_i+2k_{i+1}}$ to {0,1}. What this means is that we also need to define MLE for our operator in each gate that takes three gate label (a,b,c) and return 1 if and only if (b,c) are in-neighbor of a and gate a is addition

(resp.multiplication) gate.

As usual lets go back to layer 3. The node with +:39 is produced by adding 3 and 36 so we need to encode this such that $add(a, b, c) \to 1$. That is label of a=(0,0,1), b=(0,0,1) and c=(0,1,0) and $add([0, 0, 1], [0, 0, 1], [0, 1, 0]) \to 1$. This is an important function as it is used to determine its in-neighbor involved for addition(resp.multiplication), ie which gate from layer i+1 are responsible to produce output for gate at layer i. Similarly for +:21 $add([1, 1, 0], [0, 0, 0], [1, 1, 1]) \to 1$ other possibility is simply 0. You may have guessed it the MLE is $\widetilde{add}(x_1, x_2, x_3, y_1, y_2, y_3, z_1, z_2, z_3) = (1 - x_1)(1 - x_2)x_3(1 - y_1)(1 - y_2)y_3(1 - z_1)z_2(1 - z_3)$. Similarly define for +:21 and +:0. This is actually huge and if we used the above MLE algorithm it will be time consuming so we actually store the information of label and use the simple code below:

```
//evaluate the multilinear extension of ADD & MULT
static int OPEval(vector<int>& view,vector<int>& points,int &
   result){
    int y;
    for(size_t i=0;i<view.size();i++){
        if(view[i]==0) {
            y=1- points[i];
            if(y < 0) y+=Prime;
        } else{
            y=points[i];
        }
        result=(result * y)%Prime;
    }
    return result;
}
```

This program determines whether to use $(1 - x_i)$ or x_i based on the corresponding binary labels. The earlier program evaluates the multilinear extension of gate values at Layer i, where the values are stored sequentially. Because of this ordering, recursion can naturally generate all Lagrange basis polynomials in sequence. In contrast, the labels used in this program are not sequential. Therefore, we must explicitly store the binary labels to enable efficient evaluation. If we wanted this construction to be fully sequential, we would need$2^{k_i+2k_{i+1}}$ entries to represent all possible patterns. I leave the task of defining the multilinear extension corresponding to multiplication operations as an exercise for the reader.

With this we finally define the polynomial used in GKR where prover and verifier applies sum-check protocol to it.

$$f^i_{r_i}(b, c) = \widetilde{add_i}(r_i, b, c)(\widetilde{W}_{i+1}(b) + \widetilde{W}_{i+1}(c)) + \widetilde{mult_i}(r_i, b, c)(\widetilde{W}_{i+1}(b).\widetilde{W}_{i+1}(c))$$

$\widetilde{W}_{i+1}$ denotes the multilinear extension (MLE) of the gate values at Layer $i + 1$. The function $f^i_{r_i}(b, c)$ is conceptually analogous to $\widetilde{W}_i$, in a sense that when evaluated on Boolean inputs, it produces the value of the corresponding gate at Layer i.

At this point, I would like to caution the reader that these concepts can be difficult to absorb on a first reading. For this reason, I strongly recommend building a solid understanding of the underlying theory before focusing entirely on the implementation

details presented here. This document is primarily written from an implementation perspective.

The next section introduces significantly more complexity. Although we have already developed several important components, the construction is still incomplete. Before proceeding further, it is essential to first understand the **Sum-Check** protocol.

BFS for GKR

```
vector<int> BFS(Node* root){
    std::deque<Node*> queue;
    queue.push_back(root);
    size_t size=1;
    claim=root->value;
    std::cout<<"Claimed value of Output gate: "<<claim<<std::
       endl;
    vector<int>current;//the current gate label is bind to
       random value at first
    while(size > 0) {
        //verifier can evaluate at her own at leaf node.
        if(queue.front()->op==NONE) break;
        LabelInformation obj;
        shared_ptr<Node>witness=nullptr;
        shared_ptr<Node>helper=nullptr;
        while(size--){ // iteration layer by layer
            auto top=queue.front();
            queue.pop_front();
            switch(top->op) {
                case ADD:
                    obj.init(current,top->left.get()->label,top
                       ->right.get()->label,top->label,top->op)
                       ;
                    break;
                case MUL:
                    obj.init(current,top->left.get()->label,top
                       ->right.get()->label,top->label,top->op)
                       ;
                    break;
                case NONE: //leaf
                    break;
            };
            if(top->left) {
                if(top->left.get()->witness) continue;
                if(top->left.get()->helper) helper=top->left;
                else{
                    queue.push_back(top->left.get());
                    obj.W.push_back(top->left.get()->value);
                }
            }
            if(top->right){
                if(top->right.get()->helper) continue;
                if(top->right.get()->witness) witness=top->
```

```
                    right;
                else{
                    queue.push_back(top->right.get());
                    obj.W.push_back(top->right.get()->value);
                }
            }//Wi(r) takes wi+1(r)
        }std::cout<<std::endl; //after this we will use sum-check protocol
        if(helper){
            queue.push_front(helper.get());
            obj.W.insert(obj.W.begin(),helper.get()->value);
        }
        if(witness){
            queue.push_back(witness.get());
            obj.W.push_back(witness.get()->value);
        }
        current = SumCheck(obj);
        getchar();
        size=queue.size();
    }
    return current;
}
```

This BFS traversal is used to initiate the GKR protocol. Here, I introduce the *LabelInformation* structure, which stores the gate values at Layer i, the corresponding gate operations, as well as the gate values and labels of Layer $i+1$. For now, the `SumCheck()` method can be ignored; it will be introduced and explained in the next section.

Data structure is like this:

```
struct Little_Gate{
    vector<vector<int>>encode; //encodes parent, child
    OPERATOR op;
};
//we iterate label by label so we need to care about the gate value ADD, MULT -> its wiring predicate
// we also need to define the multilinear extension for gate value
class LabelInformation{
public:
    vector<Little_Gate>gate;//this will encode the current gate label & its neighbours
    vector<int>W;//function mapping {0,1}^k -> F where k is power of 2 since the total gate in each layer is 2^k
    LabelInformation(){}
    void init(const vector<int>& parent, string& left,string& right,string & current,OPERATOR op){
        Look_operator(gate,parent,left,right,current,op);
    }
private:
    void Look_operator(vector<Little_Gate>&value,const vector<int> parent, string& left,
```

```
        string& right,string& current,OPERATOR op){
        vector<vector<int>>array(3);//label of current and its
            child information
        array[0]=parent;
        for(size_t i=0;i<current.length();i++){
            if(current[i]=='0'){
                array[0][i]=1-array[0][i];
                if(array[0][i] < 0) array[0][i]+=Prime;
            }
        }
        for(auto &x:left) array[1].push_back((int) x-'0');
        for(auto &x:right) array[2].push_back((int) x-'0');
        Little_Gate obj;
        obj.op=op;
        obj.encode=array;
        value.push_back(obj);
    }
};
```

The `Look_operator` method is particularly important. Its first task is to convert character-based binary labels into integers, which represent the labels of the child nodes (i.e., the gate values at Layer $i+1$). We then organize this information into a 2D array structure:

- the first row stores the parent label,
- the second row stores the left-child label,
- and the third row stores the right-child label.

The parent label is handled slightly differently, as it depends on the value returned by the `SumCheck()` procedure, which will be discussed later. The `gate` variable stores all the information required for the sum-check protocol, including:

- the gate operation ($+$ or $\times$),
- the current node,
- and its in-neighbors.

As mentioned earlier, the array `W` stores the gate values corresponding to the next layer. You can simply think as "We defined the Polynomial used in GKR". If you look closely to the MLE section, we are simply capturing the data needed by these subroutine, for example evaluateW() uses W from LabelInformation and OPEval() which uses the values from gate to evaluate the MLE of $\widetilde{add_i}$ and $\widetilde{mult_i}$.

SUM-CHECK Protocol

Suppose we are given a v-variate polynomial g defined over a $\mathbb{F}$. The purpose of the sum-check protocol is for prover to provide the verifier with the following sum:

$$H := \sum_{b_1 \in \{0,1\}} \sum_{b_2 \in \{0,1\}} \cdots \sum_{b_v \in \{0,1\}} g(b_1, \cdots, b_v)$$

Summing up the evaluations of a polynomial over all Boolean inputs. It is not necessary that sum be defined over boolean inputs, we can pick a sub-field and define the sum over the sub-field element. Boolean hypercube points are superior choice in the setting of IPs and MIPs.

Description of Sum-Check Protocol.

- At the start of the protocol, the prover sends a value C_1 claimed to equal the value H defined in Equation (4.1).
- In the first round, $\mathcal{P}$ sends the univariate polynomial $g_1(X_1)$ claimed to equal

$$\sum_{(x_2,\ldots,x_v)\in\{0,1\}^{v-1}} g(X_1,x_2,\ldots,x_v).$$

  $\mathcal{V}$ checks that

$$C_1 = g_1(0)+g_1(1),$$

  and that g_1 is a univariate polynomial of degree at most $\deg_1(g)$, rejecting if not. Here, $\deg_j(g)$ denotes the degree of $g(X_1,\ldots,X_v)$ in variable X_j.
- $\mathcal{V}$ chooses a random element $r_1 \in \mathbb{F}$, and sends r_1 to $\mathcal{P}$.
- In the jth round, for $1 < j < v$, $\mathcal{P}$ sends to $\mathcal{V}$ a univariate polynomial $g_j(X_j)$ claimed to equal

$$\sum_{(x_{j+1},\ldots,x_v)\in\{0,1\}^{v-j}} g(r_1,\ldots,r_{j-1},X_j,x_{j+1},\ldots,x_v).$$

  $\mathcal{V}$ checks that g_j is a univariate polynomial of degree at most $\deg_j(g)$, and that $g_{j-1}(r_{j-1}) = g_j(0)+g_j(1)$, rejecting if not.
- $\mathcal{V}$ chooses a random element $r_j \in \mathbb{F}$, and sends r_j to $\mathcal{P}$.
- In Round v, $\mathcal{P}$ sends to $\mathcal{V}$ a univariate polynomial $g_v(X_v)$ claimed to equal

$$g(r_1,\ldots,r_{v-1},X_v).$$

  $\mathcal{V}$ checks that g_v is a univariate polynomial of degree at most $\deg_v(g)$, rejecting if not, and also checks that $g_{v-1}(r_{v-1}) = g_v(0)+g_v(1)$.
- $\mathcal{V}$ chooses a random element $r_v \in \mathbb{F}$ and evaluates $g(r_1,\ldots,r_v)$ with a single oracle query to g. $\mathcal{V}$ checks that $g_v(r_v) = g(r_1,\ldots,r_v)$, rejecting if not.
- If $\mathcal{V}$ has not yet rejected, $\mathcal{V}$ halts and accepts.

source:Thaler ; Equation (4.1) is the above equation.

Going back to our MLE of layer 3:
$\widetilde{W}(x_0,x_1,x_2) = 39(1-x_0)(1-x_1)x_2 + (1-x_0)x_1(1-x_2) + 62(1-x_0)x_1x_2 + 21x_0(1-x_1)(1-x_2) + 64x_0(1-x_1)x_2 + 21x_0x_1(1-x_2)$
Let's apply sum-check protocol:

- In first step prover sends the sum, so the claimed value is $C_1 = (39+1+62+21+64+21)\%67$ ie $C_1 \equiv 7 \pmod{67}$
  also prover sends a univariate polynomial of degree 1 which is prover response.
  Let $q_1(x) = \widetilde{W}(x,0,0) + \widetilde{W}(x,0,1) + \widetilde{W}(x,1,0) + \widetilde{W}(x,1,1)$
  $q_1(x) = 4x + 35$
  Verifier checks: $C_1 \stackrel{?}{=} q_1(0) + q_1(1)$, Verifier sends the challenge $r \in \mathbb{F}_{67}$. Lets pick 16 such that $C_2 = q_1(16) = 32$.

- The sum-check protocol is inherently adaptive, in the sense that each message sent by the prover depends on the preceding challenges issued by the verifier. At every round, the prover incorporates the verifier's random challenge into the computation and responds with the next univariate polynomial.
  Let $q_2(x) = \widetilde{W}(16, x, 0) + \widetilde{W}(16, x, 1)$
  $q_2(x) = 23x + 38$ Verifier checks: $C_2 =^? q_2(0) + q_2(1)$, picks a random challenge and send the challenge to prover and set $C_3 = q_2(r)$

- At last prover sends $q_3(x) = \widetilde{W}(16, r, x)$ verifier checks and this time it picks a random challenge s and evaluate the MLE at her own $\widetilde{W}(16, r, s)$ and checks is it equal to $C_4 = q_3(s)$

At the final step of the sum-check protocol, the verifier must evaluate the polynomial. However, in the GKR protocol, the verifier does not explicitly know $\widetilde{W}$, except at the input layer. This naturally raises the question: how can the verifier perform this evaluation?

We will address this in the next section. The univariate polynomial sent by the prover at each round is a degree-2 polynomial. A practical challenge when implementing the sum-check protocol is determining how to handle the symbolic variable x efficiently in code. Direct symbolic manipulation can quickly become cumbersome.

To simplify the implementation, we instead rely on univariate Lagrange interpolation. The key idea behind univariate Lagrange interpolation is that, given n distinct interpolation points of the form

$$\{(x_i, y_i)\}_{i=1}^{n},$$

there exists a unique polynomial of degree at most $n - 1$ that passes through all of them. Here, the x_i's are the interpolation points, while the y_i's are the corresponding evaluation values.

Phew!! There are an incredible number of smaller components involved in implementing the GKR protocol. Even the sum-check protocol alone contains many subtle details. For example, if the prover somehow knows the verifier's initial challenge in advance, it may be possible to manipulate the interaction and convince the verifier to accept a false claim.

It took me more than a month to implement this protocol, and the entire program is based on my own understanding and reasoning. At times, working through these details may seem excessive or even silly, but trust me—it teaches you a tremendous amount, especially about problem solving and connecting theoretical concepts to actual computation.

The core objective of this documentation is not merely to explain *what* the protocol is, but rather *how* it can be implemented. Most theoretical resources are excellent at explaining the "what" and the "why," but discovering the "how" is often the truly difficult part. This documentation is my attempt to bridge that gap.

Univariate Lagrange Interpolation
Given n-points of the form $\{(x_i, y_i)\}_{i=1}^{n}$ where x_i are distinct point then we can construct the polynomial of degree n-1 as:

$$q(x) = \sum_{j=1}^{n} y_j * \prod_{j=1:j!=i}^{n} \frac{x - x_j}{x_i - x_j}$$

The main advantage of this approach is that we do not need to explicitly manipulate the symbolic variable x. Instead, the prover can simply send the evaluated values of the polynomial, and the verifier can either reconstruct the polynomial using Lagrange interpolation or directly evaluate it at any point $x \in \mathbb{F}$.

Consider the polynomial

$$q_1(x) = \widetilde{W}(x, 0, 0) + \widetilde{W}(x, 0, 1) + \widetilde{W}(x, 1, 0) + \widetilde{W}(x, 1, 1).$$

We now select interpolation points x_i that are publicly known to both the prover and verifier. For simplicity, since this is a degree-1 polynomial, we choose the points 0 and 1. Using our MLE evaluation program, we evaluate the polynomial at these points. This time, the prover sends the evaluations $[35, 39]$ instead of $[4, 35]$. The verifier can then evaluate the polynomial at any random point directly using Lagrange interpolation. For example, if the verifier chooses $x = 16$, then

$$q(16) = \frac{16-1}{0-1} \cdot 35 + \frac{16-0}{1-0} \cdot 39.$$

Let's complete our sum-check by actually doing in code as:

```
#include "GKR.h"

int verifier_eval(int r,vector<int>&q)
{
    int x=(q[0]*(1-r) +q[1]*r)%67;
    return (x < 0)? 67+x:x;
}
int main()
{
    vector<int>W={0,39,1,62,21,64,21};// gate value of layer 3
    // sum-check
    vector<int>q(2);
    //first_phase
   q[0]= LabelInformation::evaluateW(W,{0,0,0})+
      LabelInformation::evaluateW(W,{0,0,1})
    +LabelInformation::evaluateW(W,{0,1,0})+LabelInformation::
       evaluateW(W,{0,1,1});
    q[1]=LabelInformation::evaluateW(W,{1,0,0})+
       LabelInformation::evaluateW(W,{1,0,1})
    +LabelInformation::evaluateW(W,{1,1,0})+LabelInformation::
       evaluateW(W,{1,1,1});
    q[0]=q[0]%67,q[1]=q[1]%67; //result in % 67
    std::cout<<"prover: ["<<q[0]<<", "<<q[1]<<"]\nverifier:"<<
       verifier_eval(16,q)<<std::endl;
    //second_phase
    q[0]=LabelInformation::evaluateW(W,{16,0,0})+
       LabelInformation::evaluateW(W,{16,0,1});
    q[1]=LabelInformation::evaluateW(W,{16,1,0})+
       LabelInformation::evaluateW(W,{16,1,1});
    q[0]=q[0]%67,q[1]=q[1]%67; //result in % 67

    std::cout<<"prover: ["<<q[0]<<", "<<q[1]<<"]\nverifier:"<<
       verifier_eval(22,q)<<std::endl;
```

```
    //third_phase
    q[0]=LabelInformation::evaluateW(W,{16,22,0});
    q[1]=LabelInformation::evaluateW(W,{16,22,1});
    q[0]=q[0]%67,q[1]=q[1]%67; //result in % 67

    std::cout<<"prover: ["<<q[0]<<" , "<<q[1]<<"]\nverifier:"<<
        verifier_eval(53,q)<<std::endl;
    //final_check
    std::cout<<"verifier: "<<LabelInformation::evaluateW(W
        ,{16,22,53});
    return 0;
}
```

```
prover: [35 , 39]
verifier:32
prover: [38 , 61]
verifier:8
prover: [6 , 2]
verifier:62
verifier: 62
```

You can verify the overall process. Here the verifier random challenges is [16,22,53] and x_i points are 0,1. In GKR we will be using 0,1,2 because of 2-degree polynomial.

Note
At this point, you may question the soundness of the protocol. For example, the prover could send $[1, 31]$ instead of $[38, 61]$, since both satisfy the condition

$$q(0) + q(1) = 32.$$

So how does the protocol remain sound?
The transcript consists of the interaction between the prover and verifier, along with the claim output. A cheating prover may attempt to fabricate claims and construct polynomials that satisfy the verifier's intermediate consistency checks. In fact, a dishonest prover can often survive several rounds of the sum-check protocol. Since the verifier reveals a random challenge at each round, the prover may evaluate its own forged polynomial at that challenge point and adapt subsequent messages accordingly to continue passing the checks.
However, the crucial observation is that the protocol ultimately terminates at the input layer, where the verifier can independently evaluate the claimed value using the public inputs(known at start). This final verification step is independent of the prover's fabricated intermediate polynomials and acts as the anchor of soundness. By the **Schwartz–Zippel lemma**, two distinct degree-d polynomials over a finite field

$\mathbb{F}$ can agree at a randomly chosen point with probability at most

$$\frac{d}{|\mathbb{F}|}.$$

In our setting, the relevant polynomial degree is 5, and the field size is 67. Therefore, the probability that a cheating prover successfully fools the verifier is at most

$$\frac{5}{67},$$

which means the probability of detecting the cheating prover at the final check is

$$1 - \frac{5}{67}.$$

Our sum-check method

```
vector<int> SumCheck(LabelInformation& obj)
{
    int row=1,column=0; //variable index so we can have a
       decision of stopping the sum-check as well as bind the
       random-challange
    auto gate=obj.gate;
    int colsize=gate[0].encode[1].size();
    for(;row <=2 ;){ //total life of sum-check
        vector<int>basis;
        basis.reserve(3);
        for(int x=0;x <3 ;x++){ // evaluation at 0,1,2
            int sum=0;
            for(size_t current=0;current<gate.size();current++)
               { //extract each gate at that layer
                sum+=Circuit::EvaluateGate(gate[current].encode
                   ,row,column,x,obj.W,gate[current].op,
                    obj.gate[current].encode);
                if( sum >= Prime) sum-=Prime;
            }
            basis.push_back(sum);
        } // prover work done .
        std::cout<<"Prover: " <<basis[0]<< ", "<<basis[1]<<", "
           <<basis[2] << std::endl;
        std::cout<<"Verifier ";//verifier
        auto response=Circuit::verifier(basis);
        if(response==-1){
            std::cout<<" Failed";
            exit(-1);
        } std::cout<<"rand: "<<response<<" g(r)= "<<claim<<std
           ::endl;
        //binding the random
        for(auto & c: gate){
            c.encode[row][column]=response;
        }
        column++;
        if(column==colsize){
```

```
31              row++;
32              column=0;
33          }
34      }
35      //at this point the verifier needs to verify the f but
           cannot on her own so both verifier & prover agree on
           line
36      //prover sends q(x) of degree at most log(n) which is gate
           at next layer. l(0)=gate[0][1] & l(1)=gate[0][2]
37      vector<int>q;
38      q.reserve(gate[0].encode[1].size()+1);//again we will send
           lagrange basis so for k -degree we need k+1 basis.
39      std::cout<<"bind:␣";
40      for(auto &x:gate[0].encode[1]) std::cout<<x<<"␣";
41      for(auto &x:gate[0].encode[2]) std::cout<<x<<"␣";
42      std::cout<<std::endl;
43      for(size_t i=0;i <= gate[0].encode[1].size();i++){
44          auto points=LabelInformation::Line(gate[0].encode[1],
               gate[0].encode[2],Prime-1-i);
45          for(auto &x:points) std::cout<<x<<"␣";
46          std::cout<<std::endl;
47          q.push_back(LabelInformation::evaluateW(obj.W,points));
48      } std::cout<<"q(x)␣=␣";
49      for(auto &x:q) std::cout <<x<<"␣";
50      //verifier can use q(0) , q(1) to evaluate the f
51      int q0=LabelInformation::Reconstruct_line(q,0),q1=
           LabelInformation::Reconstruct_line(q,1);
52      int sum=0;
53      for(size_t i=0;i< gate.size();i++){
54          sum =sum + Circuit::finalSumCheck(q0,q1,obj.gate[i].
               encode,gate[i].encode,obj.gate[i].op);
55      } sum= sum % Prime;
56      if(sum != claim) {
57          std::cerr<<"Failed␣to␣verify";
58          exit(1);
59      } //verifier is convinced that q(0)=W(l) and q(1)=W(r) : l,
           r are child label.
60      int r=dis(gen); //chooses random point
61      claim=LabelInformation::Reconstruct_line(q,r); //claim is
           reduced to next layer
62      std::cout<<"\nNext␣claim=␣"<<claim<<"␣r*␣"<<r<<std::endl;
63      return LabelInformation::Line(gate[0].encode[1],gate[0].
           encode[2],r);
64  }
```

If you observe the sum-check protocol carefully, the total communication complexity for a v-variate polynomial is $O(v)$. Recall that, in the GKR protocol, the polynomial associated with each layer requires $O(2k_{i+1})$ communication, where k_{i+1} denotes the number of bits required to label the gates at Layer $i+1$. In the implementation, the variables `row` and `column` capture this structure. Specifically:

- `gate[1]` stores the information corresponding to the left child,

- `gate[2]` stores the information corresponding to the right child.

We then directly evaluate the multilinear extension at the points 0, 1, and 2.

It is important to note that the GKR polynomial described above corresponds to only a single node. In practice, we must accumulate the contributions from every node at Layer i. The purpose of the nested loops is precisely to evaluate the multilinear extension

$$\widetilde{f}_r(b, c),$$

mirroring the recursive variable-fixing process used in the sum-check protocol.

EvaluateGate

```
static int EvaluateGate(vector<vector<int>>obj,const int& row,
   const int& column,const int& x,
    const vector<int>& W, OPERATOR op,const vector<vector<int
       >>&orignal) {
    //instead of polynomial we send evaluation at x so:
    //we need original obj value to decide the structure of
       multilinear extension for add & mult
    vector<int>view=orignal[1]; //original view such that we
       can construct multilinear extension
    int v=1;
    for(auto &x:orignal[0]) v=(v*x)%Prime;
    for(auto & x: orignal[2]) view.push_back(x);
    vector<int>points;
    obj[row][column]=x;
    for(auto & x: obj[1]) points.push_back(x);
    for(auto &x: obj[2]) points.push_back(x);

    int w1=LabelInformation::evaluateW(W,obj[1]);
    int w2=LabelInformation::evaluateW(W,obj[2]);
    int w;
    if(op==ADD){
        w=w1+w2;
        if(w > Prime) w-=Prime;
    } else if(op==MUL){
        w=(w1*w2) % Prime;
    }
    return (OPEval(view,points,v)*w) % Prime;
}
```

The variables `w1` and `w2` correspond to

$$\widetilde{W}_{i+1}(b) \quad \text{and} \quad \widetilde{W}_{i+1}(c),$$

respectively. For each node, only one operation exists: either addition or multiplication. The variable `OPEval` evaluate the multilinear extension corresponding to these operations. The purpose of `view` is to preserve the original binary labels so that the multilinear extension of the corresponding operation can be evaluated correctly. A natural question arises: what happens to the binary label of the parent node? Instead of using the original parent label directly, we replace it with the verifier's random challenge. The verifier applies a line-restriction technique to reduce the multilinear

evaluation to a single point. This is necessary because the verifier does not explicitly know

$$\widetilde{W}_{i+1}(b) \quad \text{and} \quad \widetilde{W}_{i+1}(c).$$

Rather than requesting these values individually, the verifier asks for a restriction of the form

$$W \circ l,$$

where the multilinear extension is restricted to a line. This idea will be explained in greater detail in the next section. Finally, the values of `obj[row][column]` are set to 0, 1, or 2, since we directly evaluate the multilinear extension at these points. These evaluations correspond to the y_i-values of the resulting univariate polynomial sent during the sum-check protocol.

After sending response to the verifier the verifier uses Lagrange interpolation to check the claims and evaluate the polynomial at random point.Verifier makes calls the following method

```
//except from the last step of sum check verifier just evaluate
    at f1(r)=?f2(0)+f2(1)
static int verifier(vector<int>&basis){ //-1 for fail other
   wise send the random value.
    int v=verifier_eval_at012(basis,0)+verifier_eval_at012(
       basis,1);
    if( v >= Prime) v-=Prime;
    if(v != claim) return -1;//claim is false
    //choose the random and set the next claim:
    int r=dis(gen);
    claim=verifier_eval_at012(basis,r);
    return r; //send random value
}
//lagrange interpolation to evaluate at x for a given y's & x's
    -> 0,1,2
int verifier_eval_at012(vector<int>&basis,int x){
    int a= ((x-1)*(x-2) *basis[0]) / 2;
    int b= x* (2-x) * basis[1];
    int c= (x*(x-1)*basis[2]) /2;
    int out=(a+b+c)% Prime;
    return (out < 0) ? Prime+ out : out;
}
```

This part of the implementation is fairly self-explanatory. You may notice that both the prover and verifier are implemented within the same program. If desired, the protocol can instead be simulated using a client–server architecture to achieve a cleaner separation of concerns. In that case, the primary consideration is ensuring that the verifier's random challenges are correctly bound and communicated between both parties. Other than that, the overall implementation remains largely unchanged. For simplicity, I avoided building a full interactive simulation. Instead, I directly bind the verifier's random challenges by overwriting the labels of the child nodes, which mirrors the variable-fixing process performed during the sum-check protocol.

Reducing the claim

GKR make use of Sum-check protocol to reduce a claim about layer i to layer i+1. Sum-check verifier doesn't need to know the actual polynomial($f_r(b,c)$) until the final check. The verifier can evaluate MLE $\widetilde{add_i}$ and $\widetilde{mult_i}$ but verifier cannot however evaluate $\widetilde{W}_{i+1}(b^*)$ and $\widetilde{W}_{i+1}(c^*)$ on her own without evaluating the circuit. Instead verifier asks prover to simply provide these two values, say z_1 and z_2 and uses iteration i+1 to verify that these values are as claimed. However one complication remains: the precondition for iteration i+1 is that prover claims a value for $\widetilde{W}_{i+1}(r_{i+1})$ for single point $r_{i+1} \in F^{k_{i+1}}$. So verifier needs to reduce verifying both $\widetilde{W}_{i+1}(b^*) = z_1$ and $\widetilde{W}_{i+1}(c^*) = z_2$ to verifying $\widetilde{W}_{i+1}(r_{i+1})$ at a single point in the sense that it is safe for verifier to accept the claimed values of $\widetilde{W}_{i+1}(b^*)$ and $\widetilde{W}_{i+1}(c^*)$ as long the value of $\widetilde{W}_{i+1}(r_{i+1})$ is as claimed.

let $l : F \to F^{k_{i+1}}$ be the unique line such that $l(0) = b^*$ and $l(1) = c^*$ where b^*, c^* is the verifier previous random challenge and we have to derive a line set of size k_{i+1}. Prover sends a univariate polynomial q of degree at most k_{i+1} that is claimed to be $\widetilde{W}_{i+1} \circ l$, the restriction of $\widetilde{W}_{i+1}$ to the line l. Verifier checks that $q(0) = z_1$ and $q(1) = z_2$, picks a random point $r^* \to F$, and ask prover to prove that $\widetilde{W}_{i+1}(l(r^*)) = q(r^*)$.

The interpolation points for the line are 0 and 1. Our goal is not to explicitly derive the equation of the line, such as $x + 4$ or $5x + 7$. Instead, we are interested in directly evaluating the restriction of the multilinear extension along that line.

Consider the multilinear extension

$$\widetilde{W}(x_0, x_1) = x_0(1 - x_1),$$

and suppose we restrict it to a line $l(t)$. After substitution, we obtain a degree-2 univariate polynomial, for example

$$-5x^2 - 26x - 24.$$

Again, our focus is not on symbolically manipulating this polynomial, but rather on evaluating it efficiently.

For instance, suppose we evaluate the line at

$$x \in \{18, 17, 16\}.$$

Then we directly evaluate the multilinear extension as

$$\widetilde{W}(18 + 4, 5 \cdot 18 + 7),$$

which corresponds to evaluating the resulting univariate polynomial at $x = 18$.

The reader is strongly encouraged to verify this independently, as this is one of the most elegant aspects of the construction. When the line is evaluated at some value a, that same value a becomes the interpolation point of the resulting univariate polynomial.

Since the restricted polynomial has degree 2, we require three such evaluation points. An important observation is that

$$\widetilde{W} \circ l$$

is itself a univariate polynomial. Therefore, evaluating the multilinear extension along the line automatically yields an evaluation of this univariate polynomial. The value a is the interpolation point of the univariate polynomial, not the evaluation of the line itself. With this rest of sumcheck method will be easier to understand.

line definition and Lagrange interpolation

```
 //defines the l(x) from random values again this returns the
    evaluation at p-1,p-2,... which is plug into W to send to
    verifier
static vector<int>Line(const vector<int>& l0,const vector<int>&
    l1,int x) {
    vector<int>points;
    points.reserve(l1.size());
    for(size_t i=0;i<l0.size();i++){
        int y= ((1-x)*l0[i] + x*l1[i])%Prime;
        if(y < 0) y+=Prime;
        points.push_back(y);
    }
    return points;
}
//lagrange interpolation for the line which evaluate directly
   constructing the polynomial at val
static int Reconstruct_line(const vector<int>&basis, int val) {
    vector<int>x;
    x.reserve(basis.size());
    for(size_t i=0;i < basis.size();i++) {
        x.push_back(Prime-1-i); // 66,65... size varies based
           on layer
    }
    int sum=0;
    for(size_t i=0;i<x.size();i++){
        int numerator=basis[i],denominator=1;
        for(size_t j=0;j<x.size();j++) {
            if(i==j) continue;
            numerator=(numerator*(val-x[j])) % Prime;
            denominator=(denominator*(x[i]-x[j]))% Prime;
        }
        if(numerator < 0) numerator+=Prime;
        if(denominator < 0) denominator+=Prime;
        sum=(sum + numerator * multiplicative_inverse(
           denominator)) % Prime ;
    }
    return sum;
}
```

If we evaluate the polynomial at 0 and 1 we will obviously get $\widetilde{W}_{i+1}(b^*)$ and $\widetilde{W}_{i+1}(c^*)$ respectively since $l(0) = b^*, l(1) = c^*$.

Final-sumcheck

```
static int finalSumCheck(const int& q0,const int& q1,const
   vector<vector<int>>&original,
```

```
    const vector<vector<int>>&obj, OPERATOR op){
    vector<int>view=original[1]; //original view such that we
       can construct multilinear extension
    int v=1;
    for(auto &x:original[0]) v=(v*x)%Prime;
    for(auto & x: original[2]) view.push_back(x);
    vector<int>points;
    for(auto & x: obj[1]) points.push_back(x);
    for(auto &x: obj[2]) points.push_back(x);

    int w= (op==ADD)? q0+q1 : q0*q1;
    w=w % Prime;
    return (OPEval(view,points,v) * w) % Prime;
}
```

This is simply evaluating the multivariate polynomial by verifier using q(0) and q(1) in place of $\widetilde{W}_{i+1}$. The procedure is for only one node at layer i. Recall from **sum-check** module.
If the prover is honest, then the univariate polynomial it sends is precisely $\widetilde{W}_{i+1}(l(x))$. In that case, the verifier can evaluate this polynomial at a single random point $r^* \in \mathbb{F}$, thereby reducing the claim about $\widetilde{W}_i$ to a claim about $\widetilde{W}_{i+1}$ More specifically, the value r^* corresponds to evaluating the multilinear extension at $\widetilde{W}(l(r^*))$. The prover initially binds the verifier's random challenge—which effectively replaces the parent label—with the evaluation of the line at r^*. This enables the verifier to recursively reduce the verification problem layer by layer until the input layer is reached.

This is the end of our chapter on implementing the GKR protocol. Here is the output of our program:

```
Claimed value of Output gate: 11

Prover: 11, 0, 40
Verifier rand: 66 g(r)= 6
Prover: 0, 6, 44
Verifier rand: 59 g(r)= 32
bind: 66, 59,
l(66): 6,
l(65): 13,
q(x) = 16, 5,
Next claim= 12 r* 23


Prover: 10, 2, 61
Verifier rand: 40 g(r)= 25
Prover: 25, 0, 25
Verifier rand: 19 g(r)= 60
Prover: 17, 43, 50
Verifier rand: 40 g(r)= 39
Prover: 0, 39, 48
Verifier rand: 29 g(r)= 6
bind: 40, 19, 40, 29,
l(66): 40, 9,
l(65): 40, 66,
l(64): 40, 56,
q(x) = 53, 30, 7,
Next claim= 1 r* 55


Prover: 19, 49, 42
Verifier rand: 14 g(r)= 20
Prover: 38, 49, 34
Verifier rand: 26 g(r)= 48
Prover: 38, 10, 22
Verifier rand: 65 g(r)= 13
Prover: 38, 42, 25
Verifier rand: 41 g(r)= 0
Prover: 12, 55, 5
Verifier rand: 52 g(r)= 66
Prover: 44, 22, 36
Verifier rand: 25 g(r)= 43
bind: 14, 26, 65, 41, 52, 25,
l(66): 54, 0, 38,
l(65): 27, 41, 11,
l(64): 0, 15, 51,
l(63): 40, 56, 24,
q(x) = 37, 40, 7, 11,
Next claim= 59 r* 58
```

```
Prover: 11, 48, 56
Verifier rand: 34 g(r)= 8
Prover: 9, 66, 35
Verifier rand: 44 g(r)= 4
Prover: 15, 56, 52
Verifier rand: 49 g(r)= 24
Prover: 48, 43, 62
Verifier rand: 9 g(r)= 63
Prover: 6, 57, 63
Verifier rand: 27 g(r)= 60
Prover: 66, 61, 43
Verifier rand: 34 g(r)= 40
bind: 34, 44, 49, 9, 27, 34,
l(66): 59, 61, 64,
l(65): 17, 11, 12,
l(64): 42, 28, 27,
l(63): 0, 45, 42,
q(x) = 30, 39, 38, 17,
Next claim= 42 r* 46


Prover: 10, 32, 31
Verifier rand: 35 g(r)= 26
Prover: 0, 26, 28
Verifier rand: 35 g(r)= 30
Prover: 37, 60, 52
Verifier rand: 47 g(r)= 35
Prover: 22, 13, 4
Verifier rand: 53 g(r)= 14
Prover: 16, 65, 44
Verifier rand: 51 g(r)= 30
Prover: 8, 22, 19
Verifier rand: 14 g(r)= 64
bind: 35, 35, 47, 53, 51, 14,
l(66): 17, 19, 13,
l(65): 66, 3, 46,
l(64): 48, 54, 12,
l(63): 30, 38, 45,
q(x) = 56, 48, 51, 64,
Next claim= 26 r* 35


26
```

The final value 26 is actually came from verifier where it can evaluate itself at input layer provided witness (x=17)